

Date : 6 Sep 2025

Samrat R M

samratrm2@gmail.com

Batch : DSML_Jul25_SQL_MWF_M

Note : This case study is written in BigQuery.

Section 1

Initial exploration like checking the structure & characteristics of the data :

1. Data type of all columns in the "customers" table.

```
SELECT
  column_name,
  data_type,
  is_nullable
FROM `big-query-466613.Target_DB.INFORMATION_SCHEMA.COLUMNS`
WHERE table_name = 'customers';
```

Query results Save results ▾					
Job information		Results	Visualization	JSON	Execution details
Execution graph					
Row	column_name ▾	data_type ▾	is_nullable ▾		
1	customer_id	STRING	YES		
2	customer_unique_id	STRING	YES		
3	customer_zip_code_prefix	INT64	YES		
4	customer_city	STRING	YES		
5	customer_state	STRING	YES		

Insight :

Why is primary key `customer_id` nullable ?

- BigQuery tables do not enforce primary keys or unique constraints.

2. Get the time range between which the orders were placed.

```
SELECT
  MIN(order_purchase_timestamp) AS first_order_ts ,
  MAX(order_purchase_timestamp) AS last_order_ts ,
```

```

CONCAT(
  TIMESTAMP_DIFF(
    DATE(MAX(order_purchase_timestamp)) ,
    DATE(MIN(order_purchase_timestamp)) ,
    YEAR) ,
  " years " ,
  MOD(TIMESTAMP_DIFF(
    DATE(MAX(order_purchase_timestamp)) ,
    DATE(MIN(order_purchase_timestamp)) ,
    MONTH) , 12) ,
  " months " ,
  MOD(TIMESTAMP_DIFF(
    DATE(MAX(order_purchase_timestamp)) ,
    DATE(MIN(order_purchase_timestamp)) ,
    DAY) , 30) ,
  " days "
) AS time_range_between_orders_approx
FROM `big-query-466613.Target_DB.orders` ;

```

Row	first_order_ts	last_order_ts	time_range_between_orders_ap...
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC	2 years 1 months 23 days

Insight :

- First order was placed on 2016-09-04 and last order was placed on 2018-10-17
- There is approximately 2 year 1 month and 23 days in between first and last order ever placed.
- Orders are placed from 00:00 to 23:59 Since orders are spread across all hours. We will group orders by hours and analyse.

Orders places per hour Analysis :

```

SELECT
  EXTRACT(HOUR FROM order_purchase_timestamp) AS order_hour,
  COUNT(*) AS order_count
FROM `big-query-466613.Target_DB.orders`
GROUP BY order_hour
ORDER BY order_count DESC;

```

Row	order_hour ▼	order_count ▼
1	16	6675
2	11	6578
3	14	6569
4	13	6518
5	15	6454
6	21	6217
7	20	6193
8	10	6177
9	17	6150
10	12	5995
11	19	5982
12	22	5816

```

SELECT
  SUM(CASE WHEN order_hour > 8 THEN order_count ELSE 0 END)
    AS most_busy_hours_orders ,
  SUM(CASE WHEN order_hour <= 8 THEN order_count ELSE 0 END )
    AS least_busy_hours_orders ,
  SUM(order_count)
    AS total_orders ,
  ROUND(
    100 *
    SUM(CASE WHEN order_hour <= 8 THEN order_count ELSE 0 END ) /
    SUM(order_count) ,2)
    AS most_busy_hrs_percent ,
  ROUND(

```

```

100 *
SUM(CASE WHEN order_hour > 8 THEN order_count ELSE 0 END ) /
SUM(order_count),2 )
AS last_busy_hrs_percent
FROM
(SELECT
  EXTRACT(HOUR FROM order_purchase_timestamp) AS order_hour,
  COUNT(*) AS order_count
FROM `big-query-466613.Target_DB.orders`
GROUP BY order_hour) t

```

Query results

[Save results ▾](#)

Job information **Results** Visualization JSON Execution details Execution graph

Row	most_busy_hours...	least_busy_hours...	total_orders ▾	most_busy_hrs_p...	last_busy_hrs_per...
1	90001	9440	99441	9.49	90.51

Key Insights :

- Most of the orders are placed during **9:00 – 23:00**
Orders range from **6675 – 4123** per hour
90.5% orders are placed during these hours.
- Least orders are placed during **00:00 – 08:00**
Orders range from **2967 – 188** per hour
Only 9.5% of order are placed during these hours.

Action Recommendations :

- Employees work timing should be schedules according to these hours.
- Restocks and website maintenance should be done during the least busy hours **00:00 – 08:00** .

3. Count the Cities & States of customers who ordered during the given period.

```

SELECT
  COUNT(DISTINCT c.customer_city) city_count ,
  COUNT(DISTINCT c.customer_state ) AS state_count
FROM `Target_DB.orders` o
INNER JOIN `Target_DB.customers` c
  ON o.customer_id = c.customer_id ;

```

Query results

Job information

Results

Visualization

Row	city_count	state_count
1	4119	27

Key Insights :

- Customers have ordered from 27 different states and 4119 different cities.
- **Order date was not specified** so, this data is all ordered in the orders table.

State Date Analysis :

```
SELECT
  t.state ,
  t.city_count_per_state,
  t.city_count_perc ,
  SUM(t.city_count_perc) OVER(ORDER BY  t.city_count_per_state DESC) AS moving_s
FROM
  (SELECT
    c.customer_state AS state ,
    COUNT(DISTINCT c.customer_city) city_count_per_state ,
    ROUND(100 * COUNT(DISTINCT c.customer_city)/(SUM(COUNT(DISTINCT c.customer_ci
FROM `Target_DB.orders` o
INNER JOIN `Target_DB.customers` c
  ON o.customer_id = c.customer_id
GROUP BY 1) t
```

Row	state ▼	city_count_per_st...	city_count_perc ▼	moving_sum_city...
1	MG	745	17.29	17.29
2	SP	629	14.59	31.88
3	RS	379	8.79	40.67
4	PR	364	8.45	49.12
5	BA	353	8.19	57.31
6	SC	240	5.57	62.88
7	GO	178	4.13	67.01
8	CE	161	3.74	70.75
9	PE	152	3.53	74.28
10	RJ	149	3.46	77.74
11	MA	122	2.83	80.57
12	MT	101	2.34	82.91

Key Insights :

Out of the 27 states.

- First 4 states contribute to 49% of the orders.
- First 11 states contribute to 80.5% of the orders.

Action Recommendations :

- Target ADs and offers to these 11 states with highest orders.
- Create priority categories for states.
Eg : First 4 states P0 , States Ranked from 4 to 11 states P1 etc..

Section 2 :

In-depth Exploration:

1. Is there a growing trend in the no. of orders placed over the past years?

```
WITH orders_count_per_year AS (
  SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS year ,
    COUNT(DISTINCT EXTRACT(MONTH FROM DATE(order_purchase_timestamp))) AS months
    COUNT(*) AS order_count
  FROM `Target_DB.orders`
  GROUP BY 1
) ,
```

```

orders_with_prev AS (
  SELECT
    year ,
    months ,
    order_count ,
    LAG(order_count) OVER(ORDER BY year) AS prev_yr_orders
  FROM orders_count_per_year
  ORDER BY 1
)

SELECT
  year ,
  months AS active_months ,
  order_count AS total_order_cnt ,
  CONCAT(
    ROUND( 100 * (order_count - prev_yr_orders)/prev_yr_orders ,2)
    , "%" )
  AS yoy_orders_growth_prec
FROM orders_with_prev

```

Query results

Job information		Results	Visualization	JSON	Execution details
Row	year ▼	active_months ▼	total_order_cnt ▼	yoy_orders_growth_prec ▼	
1	2016	3	329	<i>null</i>	
2	2017	12	45101	13608.51%	
3	2018	10	54011	19.76%	

Key Insights :

- Since there are only 3 years and the first year has only ~300 orders.
Increase in orders between first year and second year is not meaningful.
- From 2017-2018 there is a ~19% increase in orders and 2018 only has 10 active months with last month being 2018-10-17 So, the 2018 orders data is still expected to increase and so does the growth percentage.

2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

```

WITH orders_count_grp_by_mnt_yr AS (
  SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS year ,
    EXTRACT(MONTH FROM DATE(order_purchase_timestamp)) AS months ,
    COUNT(*) AS order_count

```

```

FROM `Target_DB.orders`
GROUP BY 1 , 2
ORDER BY 1, 2
) ,

total_orders_per_month AS (
    SELECT
        months AS month ,
        COUNT(DISTINCT year) AS month_count,
        SUM(order_count) AS total_orders
    FROM orders_count_grp_by_mnt_yr
    GROUP BY 1
) ,

valid_orders_per_month AS (
    SELECT
        months AS month ,
        COUNT(DISTINCT year) AS month_count,
        SUM(IF(order_count < 20 , 0 , order_count)) AS valid_month_order_total
    FROM orders_count_grp_by_mnt_yr
    GROUP BY 1
) ,

average_order_tbl AS (
    SELECT
        month ,
        FORMAT_DATE("%B",DATE(2016,month,1)) AS month_name ,
        ROUND(valid_month_order_total/month_count) AS avg_orders
    FROM valid_orders_per_month
) ,

prev_month_data AS
(SELECT
    month ,
    month_name ,
    avg_orders ,
    LAG(avg_orders) OVER(ORDER BY month) AS prev_month_avg
FROM average_order_tbl
ORDER BY month) ,

final_table AS (
    SELECT
        month_name ,
        avg_orders ,
        ROUND(100 * (avg_orders - prev_month_avg ) / prev_month_avg , 2 )
        AS MoM_growth_perc ,
        ROUND(AVG(avg_orders) OVER(),2) AS avg_order_per_month ,
        ROUND(
            100 *
            (avg_orders - AVG(avg_orders) OVER())
            /AVG(avg_orders) OVER(),2)
        AS diff_from_avg_perc
    FROM prev_month_data

```



```

ORDER BY month
)

SELECT
    month_name ,
    avg_orders ,
    CONCAT(MoM_growth_perc, "%") AS MoM_growth_perc ,
    avg_order_per_month ,
    CONCAT(diff_from_avg_perc , "%") AS diff_from_avg_perc
FROM final_table ;

```

Metrics Definition :

- `month_name` Name of that month
- `avg_orders` average order of that specific month.
- `MoM_growth_perc` growth in percentage form the pervious month
- `avg_order_per_month` Average of orders all months (average of `avg_orders`)
- `diff_from_avg_perc` Difference between `avg_orders` and `avg_order_per_month`

Row	month_name	avg_orders	MoM_growth_perc	avg_order_per_m...	diff_from_avg_perc
1	January	4035.0	null	4328.58	-6.78%
2	February	4254.0	5.43%	4328.58	-1.72%
3	March	4947.0	16.29%	4328.58	14.29%
4	April	4672.0	-5.56%	4328.58	7.93%
5	May	5287.0	13.16%	4328.58	22.14%
6	June	4706.0	-10.99%	4328.58	8.72%
7	July	5159.0	9.63%	4328.58	19.18%
8	August	5422.0	5.1%	4328.58	25.26%
9	September	1428.0	-73.66%	4328.58	-67.01%
10	October	1652.0	15.69%	4328.58	-61.84%
11	November	7544.0	356.66%	4328.58	74.28%
12	December	2837.0	-62.39%	4328.58	-34.46%

Insight :

MoM growth data on its own is misleading. So, we use average of orders across all month to validate our findings.

Eg :November has a MoM growth of 350% but **its inflated because of previous month data**. Its only 74% higher than average per month.

- September and December shows significant drop in sales across both MoM growth and average per month.
- March, May, July, November Shows atleast 10% increase in orders across both MoM growth and average per month.

- November has the highest sales increase with 350% MoM growth and 74% higher than average.

Query Explanation :

Why use `avg_orders` instead of `total_orders` ?

Row	year	months	order_count
1	2016	9	4
2	2016	10	324
3	2016	12	1
4	2017	1	800
5	2017	2	1780
6	2017	3	2682
7	2017	4	2404

`orders_count_grp_by_mnt_yr` from this table we can see that there are some months with less than 20 orders

Row	month	month_cnt	total_orders
1	2	2	8508
2	4	2	9343
3	9	3	4305
4	10	3	4959
5	1	2	8069

`total_orders_per_month` from this table we can see that there some active months that appear 3 times while some months appear only 1 time.

New conditions are added for these two reasons (mentioned above) :

In table `valid_orders_per_month`

- Filter out months with less than 20 orders. `valid_month_order_total`
- Instead of total orders across all years. We focus average orders per month to find seasonal trend across months. `avg_orders`

3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

```
SELECT
CASE
  WHEN hour >= 0 AND hour <= 6 THEN "Dawn"
  WHEN hour >= 7 AND hour <= 12 THEN "Mornings"
  WHEN hour >= 12 AND hour <= 18 THEN "Afternoon"
  WHEN hour >= 18 AND hour <= 23 THEN "Night"
END AS time_of_day ,
COUNT(*) AS order_count ,
CONCAT(ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 2),"%")
  AS time_of_day_order_percent
FROM (
  SELECT
    EXTRACT(HOUR FROM TIME(order_purchase_timestamp)) AS hour
  FROM `Target_DB.orders`) t
GROUP BY 1 ;
```

Row	time_of_day ▼	order_count ▼	time_of_day_order_percent ▼
1	Afternoon	38135	38.35%
2	Dawn	5242	5.27%
3	Mornings	27733	27.89%
4	Night	28331	28.49%

Key Insights :

- 38% of orders are placed during the Afternoon.
- Only 5% of orders are placed during the Dawn.

Section 3 :

(Evolution of E-commerce orders in the Brazil region)

1. Get the month on month no. of orders placed in each state.

```

WITH cust_state_orders_per_month AS (
  SELECT
    c.customer_state ,
    DATE_TRUNC(DATE(o.order_purchase_timestamp), MONTH) AS ym,
    COUNT(*) AS order_count
  FROM `Target_DB.orders` o
  INNER JOIN `Target_DB.customers` c
    ON o.customer_id = c.customer_id
  GROUP BY 1, 2
) ,

prev_month_data AS
(SELECT
  customer_state ,
  ym ,
  order_count ,
  LAG(order_count , 1) OVER(PARTITION BY customer_state ORDER BY ym) AS prev_mo
FROM
  cust_state_orders_per_month
)

SELECT
  customer_state,
  FORMAT_DATE('%B %Y', ym) AS year_month,
  order_count,
  prev_month_order_count,
  IF(
    prev_month_order_count IS NULL , NULL ,      -- first month for that state
    ROUND(100 * SAFE_DIVIDE(order_count - prev_month_order_count, prev_month_order_count)
  ) AS MoM_growth_perc
FROM prev_month_data
ORDER BY customer_state, ym;

```

Row	customer_state ▼	month_name ▼	order_count ▼	prev_month_order...	MoM_growth_perc ▼
1	AC	January 2017	2	null	null
2	AC	February 2017	3	2	50.0
3	AC	March 2017	2	3	-33.33
4	AC	April 2017	5	2	150.0
5	AC	May 2017	8	5	60.0
6	AC	June 2017	4	8	-50.0
7	AC	July 2017	5	4	25.0
8	AC	August 2017	4	5	-20.0
9	AC	September 2017	5	4	25.0
10	AC	October 2017	6	5	20.0

Key Insights :

- There is a total of 565 states with monthly orders from 2016-2018

- Bottom 250+ of them have order count less than 50
Which means in that state customers placed less than 50 orders in a month.
- Top 45 have order count more than 500

There is a big deviation between the top and the bottom state orders per month. So, I have categorised them into two groups.

Whale :

Insight Query :

```
SELECT
  whale_count,
  whale AS total_orders_from_whale ,
  ROUND(100 * (whale_count / total) ,2) AS whale_perc ,
  ROUND(100* (whale/total_oc)) AS whale_contribution ,
  ROUND(whale / whale_count ,2) AS whale_order_avg
FROM
  (SELECT
    SUM(CASE WHEN order_count >= 500 THEN order_count ELSE 0 END) AS whale ,
    SUM(CASE WHEN order_count >= 500 THEN 1 ELSE 0 END) AS whale_count ,
    COUNT(*) AS total,
    SUM(order_count) AS total_oc
  FROM final_table ) t
```

✓ This query will process 7.59 MB when run.

Query results

 Save

Job information	Results	Visualization	JSON	Execution details	Execution (
Row	whale_count	total_orders_from...	whale_perc	whale_contribution	whale_order_avg
1	45	61008	7.96	61.0	1355.73

"Whales" the states-month with orders **more than 500 per month** :

- They are top 7% of the total state-month population.
- They contribute to 60% of the total orders
- With an average order value of 1355 per month.

Similarly,

Small fish :

```

SELECT
  small_fish_count,
  small_fish AS total_orders_from_small_fish ,
  ROUND(100 * (small_fish_count / total) ,2) AS small_fish_perc ,
  ROUND(100* (small_fish/total_oc)) AS small_fish_contribution ,
  ROUND(small_fish / small_fish_count ,2) AS small_fish_order_avg
FROM
  (SELECT
    SUM(CASE WHEN order_count < 50 THEN order_count ELSE 0 END) AS small_fish ,
    SUM(CASE WHEN order_count < 50 THEN 1 ELSE 0 END) AS small_fish_count ,
    COUNT(*) AS total,
    SUM(order_count) AS total_oc
  FROM final_table ) t

```

✓ This query will process 7.59 MB when run.

Query results

[Save](#)

Job information	Results	Visualization	JSON	Execution details	Execution gr
Row	small_fish_count	total_orders_from...	small_fish_perc	small_fish_contri...	small_fish_order_...
1	318	5644	56.28	6.0	17.75

"small fish" the states-month with orders **less than 50 per month** :

- The are bottom 56% of the total state-month population.
- They contribute to 6% of the total orders
- With an average order value of ~18 per month.

Action Recommendations :

- Marketing team should target high-growth pockets
- Operations team should scale logistics where demand is rising
- MoM orders data insight will help organisation leaders to spot structural shifts in regional demand early.

2. How are the customers distributed across all the states?

```

WITH state_orders AS (
  SELECT
    c.customer_state,
    COUNT(*) AS orders_count
  FROM `Target_DB.customers` c

```

```

JOIN `Target_DB.orders` o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_state
),

total_orders AS (
  SELECT SUM(orders_count) AS total_orders
  FROM state_orders
)

SELECT
  s.customer_state,
  s.orders_count,
  t.total_orders,
  ROUND(100 * s.orders_count / t.total_orders, 2) AS pct_of_total_orders,
  ROUND(
    SUM(100 * s.orders_count / t.total_orders)
    OVER (ORDER BY s.orders_count DESC
          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW),
    2
  ) AS mvg_sum_pct_total_orders
FROM state_orders s
CROSS JOIN total_orders t
ORDER BY s.orders_count DESC;

```

Row	customer_state ▼	orders_count ▼	total_orders ▼	pct_of_total_orders ▼	mvg_sum_pct_tot...
1	SP	41746	99441	41.98	41.98
2	RJ	12852	99441	12.92	54.9
3	MG	11635	99441	11.7	66.61
4	RS	5466	99441	5.5	72.1
5	PR	5045	99441	5.07	77.18
6	SC	3637	99441	3.66	80.83
7	BA	3380	99441	3.4	84.23
8	DF	2140	99441	2.15	86.38
9	ES	2033	99441	2.04	88.43
10	GO	2020	99441	2.03	90.46

Key Insights :

- State **SP** has the highest customer base of ~42%
- Top 10 states out of 27 contribute to 90% of the customers.
- The bottom 10 states contribute to <4% of the customers.
- Classic 80/20 distribution. Top 6 states contribute to 80% of customers.
- Rank 11-17 contribute to ~7% and can be potential opportunity zones.

Action Recommendations :

- Focus marketing and logistics on the top 6 states where ROI will be highest.

- Run pilots , campaigns and AB testing on the states ranked from 7 to 17 to find growth opportunities.
 - Optimise cost and operations in low volume states.
-

Section 4 :

(Impact on Economy)

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only)

```
WITH order_price_table AS (  
    SELECT  
        order_id,  
        SUM(price) AS total_price,  
        SUM(freight_value) AS total_freight,  
        SUM(price) + SUM(freight_value) AS order_total  
    FROM `Target_DB.order_items`  
    GROUP BY order_id  
) ,  
  
cost_of_orders_table AS  
(  
    SELECT  
        EXTRACT(YEAR FROM DATE(o.order_purchase_timestamp )) AS year ,  
        ROUND(SUM(oi.order_total),2) AS cost_of_orders  
    FROM `Target_DB.orders` o  
    INNER JOIN order_price_table oi  
        ON oi.order_id = o.order_id  
    WHERE  
        EXTRACT(YEAR FROM DATE(o.order_purchase_timestamp )) IN (2017 , 2018)  
        AND EXTRACT(MONTH FROM DATE(o.order_purchase_timestamp )) BETWEEN 1 AND 8  
    GROUP BY 1)  
  
SELECT  
    year,  
    cost_of_orders,  
    ROUND(  
        100 * (  
            cost_of_orders - LAG(cost_of_orders) OVER (ORDER BY year) /  
            LAG(cost_of_orders) OVER (ORDER BY year)  
        ),  
        2  
    ) AS YoY_growth  
FROM cost_of_orders_table  
ORDER BY year;
```


Query completed				
Query results				
Job information		Results	Visualization	JSON
			Exec	
Row	year	cost_of_orders	YoY_growth	
1	2017	3610270.15	null	
2	2018	8643531.14	139.42	

Key Insights :

Explosive Growth between Jan–Aug 2017 and Jan–Aug 2018, the total cost of orders increased by **139%**, meaning the e-commerce market of Target, Brazil more than doubled in size within a single year.

Action Recommendations :

This rapid growth signals a sharp increase in market share and strong momentum. To sustain it, stakeholders should prioritise on 4 things to support the surge in demand.

- Scaling **operations**
- Strengthening **infrastructure**
- Expanding the **product catalog**
- Investing in **employees**

2. Calculate the Total & Average value of order price for each state.

```
WITH order_price_table AS (
  SELECT
    order_id,
    SUM(price) AS total_price,
    SUM(freight_value) AS total_freight,
    SUM(price) + SUM(freight_value) AS order_total
  FROM `Target_DB.order_items`
  GROUP BY order_id
)

SELECT
  c.customer_state ,
```

```

ROUND(SUM(oi.order_total),2) AS total_order_price ,
ROUND(AVG(oi.order_total),2) AS avg_order_price ,
ROUND
  ((SUM(oi.order_total) / SUM(SUM(oi.order_total)) OVER() ) * 100
  , 2 )
  AS total_order_price_perc
FROM
  (SELECT order_id , customer_id
  FROM `Target_DB.orders`
  WHERE order_status = "delivered") o
  -- only for orders that has been delivered
INNER JOIN order_price_table oi
  ON o.order_id = oi.order_id
INNER JOIN `Target_DB.customers` c
  ON c.customer_id = o.customer_id
GROUP BY c.customer_state
ORDER BY 4 DESC ;

```

Row	customer_state ▼	total_order_price ▼	avg_order_price ▼	total_order_price_... ▼
1	SP	5769703.15	142.46	37.42
2	RJ	2055401.57	166.43	13.33
3	MG	1818891.67	160.2	11.8
4	RS	861472.79	161.17	5.59
5	PR	781708.8	158.79	5.07
6	SC	595127.78	167.83	3.86
7	BA	591137.81	181.55	3.83
8	DF	346123.35	166.41	2.24
9	GO	334212.35	170.78	2.17
10	ES	317657.93	159.23	2.06

Key Insights :

This table is almost identical to the Problem 2 in Section 3. Where we calculate the total orders count for each state and compare them. The insights are almost identical since order count and coder price count are directly proportional.

- State **SP** has the highest customer base of ~37%
- Top 10 states out of 27 contribute to ~85% of the customers.
- The bottom 10 states contribute to <5% of the customers.
- Classic 80/20 distribution. Top 6 states contribute to ~80% of customers.
- Rank 11-17 contribute to ~9% and can be potential opportunity zones.

Action Recommendations :

It should be same as mentioned in Problem 2 in Section 3 (How are the customers distributed across all the states) because "action recommendations" for order_count and order_price are same.

3. Calculate the Total & Average value of order freight for each state.

```
WITH order_price_table AS (  
  SELECT  
    order_id,  
    SUM(price) AS total_price,  
    SUM(freight_value) AS total_freight,  
    SUM(price) + SUM(freight_value) AS order_total  
  FROM `Target_DB.order_items`  
  GROUP BY order_id  
)  
  
SELECT  
  c.customer_state ,  
  ROUND(SUM(oi.total_freight),2) AS total_freight_price ,  
  ROUND(AVG(oi.total_freight),2) AS avg_total_freight_price ,  
  ROUND  
    ((SUM(oi.total_freight) / SUM(SUM(oi.total_freight)) OVER() ) * 100  
    , 2 )  
  AS total_total_freight_price_perc  
FROM  
  (SELECT order_id , customer_id  
  FROM `Target_DB.orders`  
  WHERE order_status = "delivered") o  
  -- only for orders that has been delivered  
INNER JOIN order_price_table oi  
  ON o.order_id = oi.order_id  
INNER JOIN `Target_DB.customers` c  
  ON c.customer_id = o.customer_id  
GROUP BY c.customer_state  
ORDER BY 4 DESC ;
```

Row	customer_state	total_freight_price	avg_total_freight_...	total_total_freight_price_perc
1	SP	702069.99	17.33	31.94
2	RJ	295750.44	23.95	13.45
3	MG	266409.84	23.46	12.12
4	RS	132575.32	24.8	6.03
5	PR	115645.29	23.49	5.26
6	BA	97553.67	29.96	4.44
7	SC	88115.65	24.85	4.01
8	PE	57082.56	35.83	2.6
9	GO	51375.65	26.25	2.34
10	DF	49624.94	23.86	2.26

Key Insights :

- Insights follow the same trend as the previous problem (Total & Average value of order freight for each state) because price of order , order count and freight price are directly proportional.

Section 5

(Analysis based on sales, freight and delivery time)

1. Find the no. of days taken to deliver each order from the order's purchase date as delivery time. Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

```
SELECT
    DATE_DIFF(order_delivered_customer_date , order_purchase_timestamp , DAY)
        AS time_to_deliver ,
    DATE_DIFF(order_estimated_delivery_date , order_purchase_timestamp, DAY)
        AS estimated_delivery ,
    DATE_DIFF(order_delivered_customer_date , order_estimated_delivery_date, DAY)
        AS diff_estimated_delivery
FROM `Target_DB.orders`
WHERE order_status = "delivered" ;
```

Row	time_to_deliver	estimated_delivery	diff_estimated_de...
1	54	18	36
2	43	50	-6
3	36	50	-14
4	29	50	-20
5	30	50	-19
6	44	50	-5
7	68	50	18
8	47	50	-2
9	43	50	-7
10	43	50	-7

Insights Query :

```

WITH final_table AS (
  SELECT
    DATE_DIFF(order_delivered_customer_date , order_purchase_timestamp , DAY)
      AS time_to_deliver ,
    DATE_DIFF(order_estimated_delivery_date , order_purchase_timestamp, DAY)
      AS estimated_delivery ,
    DATE_DIFF(order_delivered_customer_date , order_estimated_delivery_date, DAY)
      AS diff_estimated_delivery
  FROM `Target_DB.orders`
  WHERE order_status = "delivered" ) ,

delivery_count_table AS
(SELECT
  SUM(CASE WHEN diff_estimated_delivery > 0 THEN 1 ELSE 0 END )
    AS late_delivery_cnt ,
  SUM(CASE WHEN diff_estimated_delivery = 0 THEN 1 ELSE 0 END )
    AS on_time_delivery_cnt ,
  SUM(CASE WHEN diff_estimated_delivery < 0 THEN 1 ELSE 0 END )
    AS early_delivery_cnt ,
  COUNT(*) AS total_orders_delivered ,
  SUM(CASE WHEN diff_estimated_delivery > 0 THEN diff_estimated_delivery ELSE 0
    AS sum_late_delivery_days ,
  SUM(CASE WHEN diff_estimated_delivery < 0 THEN diff_estimated_delivery ELSE 0
    AS sum_early_delivery_days ,
  FROM final_table ) ,

```

```
percentage_table AS
(SELECT
  ROUND((late_delivery_cnt / total_orders_delivered ) * 100 , 2)
  AS late_delivery_perc ,
  ROUND((on_time_delivery_cnt / total_orders_delivered ) * 100 , 2)
  AS on_time_delivery_perc ,
  ROUND((early_delivery_cnt / total_orders_delivered ) * 100 , 2)
  AS early_delivery_perc ,
  ROUND(sum_late_delivery_days / late_delivery_cnt ,2)
  AS avg_late_delivery_days ,
  ROUND(ABS(sum_early_delivery_days / early_delivery_cnt) ,2)
  AS avg_early_delivery_days ,
FROM delivery_count_table)
```

Key Insights :

Query results Save results					
Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	late_delivery_perc	on_time_delivery_perc	early_delivery_perc	avg_late_delivery_days	avg_early_delivery_days
1	6.77	2.85	90.36	10.62	12.92
Row	late_delivery_cnt	on_time_delivery...	early_delivery_cnt	total_orders_delivered	
1	6534	2754	87182	96478	

Delivery	Early delivery	On time Delivery	Late Delivery	Total Delivery
Count	87182	2754	6534	96478
Percentage	90.36%	2.85%	6.77%	
Average Days	10.62	17.56	12.92	12.09

- Average delivery time of an order is 12 days.
- Average estimated delivery time of an order is 23 days
So,
- 90% of orders are delivered early and they are early by ~11 days.
- 3% of orders are delivered on the estimated date.
- 7% orders are delivered late by ~13 days from the estimated date.

2. Find out the top 5 states with the highest & lowest average freight value

```
WITH order_items_table AS
(SELECT
```

```

        order_id ,
        SUM(freight_value) AS freight_total ,
    FROM `Target_DB.order_items`
    GROUP BY order_id ) ,

```

```

avg_freight_value_per_state AS
    (SELECT
        c.customer_state ,
        ROUND(AVG(oi.freight_total),2)
        AS avg_freight_value_per_order
    FROM `Target_DB.orders` o
    INNER JOIN order_items_table oi
        ON o.order_id = oi.order_id
    INNER JOIN `Target_DB.customers` c
        ON c.customer_id = o.customer_id
    GROUP BY 1 ) ,

```

```

ranked_states_tbl AS (
    SELECT
        customer_state ,
        avg_freight_value_per_order ,
        DENSE_RANK()
            OVER(ORDER BY avg_freight_value_per_order ) AS lowest_freight_rnk ,
        DENSE_RANK()
            OVER(ORDER BY avg_freight_value_per_order DESC ) AS highest_freight_rnk ,
    FROM avg_freight_value_per_state
)

```

```

SELECT
    customer_state,
    avg_freight_value_per_order,
    CASE
        WHEN lowest_freight_rnk <= 5 THEN 'Lowest (Top 5)'
        WHEN highest_freight_rnk <= 5 THEN 'Highest (Top 5)'
    END AS category,
    CASE
        WHEN lowest_freight_rnk <= 5 THEN lowest_freight_rnk
        ELSE highest_freight_rnk
    END AS category_rank
FROM ranked_states_tbl
WHERE lowest_freight_rnk <= 5 OR highest_freight_rnk <= 5
ORDER BY category, category_rank;

```

Row	customer_state	avg_freight_value_per_order	category	category_rank
1	RR	48.59	Highest (Top 5)	1
2	PB	48.35	Highest (Top 5)	2
3	RO	46.22	Highest (Top 5)	3
4	AC	45.52	Highest (Top 5)	4
5	PI	43.04	Highest (Top 5)	5
6	SP	17.37	Lowest (Top 5)	1
7	MG	23.46	Lowest (Top 5)	2
8	PR	23.58	Lowest (Top 5)	3
9	DF	23.82	Lowest (Top 5)	4
10	RJ	23.95	Lowest (Top 5)	5

Key Insights :

- States with lowest average freights value per order is in range 17-23.
Also the states with highest order count and a combined contribution of >70% of total orders in Target Brazil (from Section 4 Q2).
- States with highest average freights value per order is in range 43-48.
The freight value gap (17 VS 48) means that some states charge freights cost that are almost 3x more expensive. Reducing this significant gap can increase customer satisfaction and unlock growth potential.

Action Recommendations :

Higher freight cost may discourage customers from placing orders.

- Analytics team** should identify states with high growth potential and high freight costs.
- Logistics teams** should focus on decreasing the freights value for states that have higher growth potential.

3. Find out the top 5 states with the highest & lowest average delivery time.

```
WITH order_delivery_table AS (
  SELECT
    order_id,
    customer_id,
    DATE_DIFF(
      DATE(order_delivered_customer_date),
      DATE(order_purchase_timestamp),
      DAY)
    AS delivery_time
  FROM `Target_DB.orders`
```



```

WHERE order_status = 'delivered'
      AND order_delivered_customer_date IS NOT NULL
      AND order_purchase_timestamp IS NOT NULL
),

avg_delivery_per_state AS (
  SELECT
    c.customer_state,
    ROUND(AVG(o.delivery_time), 2)
      AS avg_delivery_time
  FROM order_delivery_table o
  JOIN `Target_DB.customers` c
    ON c.customer_id = o.customer_id
  GROUP BY c.customer_state
),

ranked_states AS (
  SELECT
    customer_state,
    avg_delivery_time,
    DENSE_RANK()
      OVER (ORDER BY avg_delivery_time ASC)
      AS rank_fastest,
    DENSE_RANK()
      OVER (ORDER BY avg_delivery_time DESC)
      AS rank_slowest
  FROM avg_delivery_per_state
)

SELECT
  customer_state,
  avg_delivery_time,
  CASE
    WHEN rank_fastest <= 5 THEN 'Fastest (Top 5)'
    WHEN rank_slowest <= 5 THEN 'Slowest (Top 5)'
  END AS category,
  CASE
    WHEN rank_fastest <= 5 THEN rank_fastest
    ELSE rank_slowest
  END AS category_rank
FROM ranked_states
WHERE rank_fastest <= 5 OR rank_slowest <= 5
ORDER BY category, category_rank;

```

Row	customer_state	avg_delivery_time	category	category_rank
1	SP	8.7	Fastest (Top 5)	1
2	MG	11.94	Fastest (Top 5)	2
3	PR	11.94	Fastest (Top 5)	2
4	DF	12.9	Fastest (Top 5)	3
5	SC	14.9	Fastest (Top 5)	4
6	RJ	15.24	Fastest (Top 5)	5
7	RR	29.34	Slowest (Top 5)	1
8	AP	27.18	Slowest (Top 5)	2
9	AM	26.36	Slowest (Top 5)	3
10	AL	24.5	Slowest (Top 5)	4
11	PA	23.73	Slowest (Top 5)	5

Freights Value and Delivery time for states

Key Insights :

Reference : Table with states and their orders data :

Row	customer_state	orders_count	total_orders	pct_of_total_orders	mvg_sum_pct_tot...
1	SP	41746	99441	41.98	41.98
2	RJ	12852	99441	12.92	54.9
3	MG	11635	99441	11.7	66.61
4	RS	5466	99441	5.5	72.1
5	PR	5045	99441	5.07	77.18
6	SC	3637	99441	3.66	80.83
7	BA	3380	99441	3.4	84.23
8	DF	2140	99441	2.15	86.38
9	ES	2033	99441	2.04	88.43
10	GO	2020	99441	2.03	90.46

- Freight and delivery time analysis shows that some high-volume states are not among the fastest or cheapest to serve.

For example : **RS** contributes ~5.5% of the total orders in Brazil but does not appear in top 5 fastest delivering states or lowest freight states.

This implies the logistics performance is not aligned with business importance.

Action Recommendations :

- Prioritise logistics optimisation in high-order states like RS — customers here are numerous, so even small improvements have outsized impact.
- Cross team collaboration :
 - Analytics team to regularly surface such mismatches.
 - Logistics team to fix them.
 - Marketing/Strategy team to monitor customer satisfaction in those regions.

4. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

```
WITH order_delivery_table AS
(
  SELECT
    customer_id ,
    DATE_DIFF(order_delivered_customer_date , order_purchase_timestamp , DAY)
      AS time_to_deliver ,
    DATE_DIFF(order_estimated_delivery_date , order_purchase_timestamp, DAY)
      AS estimated_delivery
  FROM `Target_DB.orders`
  WHERE order_status = "delivered" )

SELECT
  c.customer_state AS states_with_fast_delivery,
  ROUND(AVG(od.estimated_delivery - od.time_to_deliver) ,2) AS avg_days_early
FROM order_delivery_table od
INNER JOIN `Target_DB.customers` c
  ON c.customer_id = od.customer_id
GROUP BY 1
HAVING AVG(od.estimated_delivery - od.time_to_deliver) > 0
ORDER BY 2 DESC
LIMIT 5;
```

Row	states_with_fast_delivery ▼	avg_days_early ▼
1	AC	20.09
2	RO	19.47
3	AP	19.13
4	AM	18.94
5	RR	16.66

Key Insights :

- Not one of the top 10 high-volume order contributors are in the top 5 fastest delivering states. (Reference : Table with states and their orders data - Section 3 Q2)
- The average days early for states range from 20 – 8 and No states has a average delivery time that is greater than the estimated time.

Action Recommendations :

- Logistics team should focus on increasing the delivery time of high-volume order contributors states.
- States that deliver very early (15+ days ahead) may have overly conservative estimates and might be a signal to recalibrate.

Section 6 :

(Analysis based on the payments)

1. Find the month on month no. of orders placed using different payment types.

```
SELECT
  DATE_TRUNC DATE(o.order_purchase_timestamp), MONTH) AS ym,
  SUM(CASE WHEN p.payment_type = "UPI" THEN 1 ELSE 0 END) AS UPI ,
  SUM(CASE WHEN p.payment_type = "credit_card" THEN 1 ELSE 0 END) AS Credit_card
  SUM(CASE WHEN p.payment_type = "debit_card" THEN 1 ELSE 0 END) AS Debit_card ,
  SUM(CASE WHEN p.payment_type = "voucher" THEN 1 ELSE 0 END) AS Voucher
FROM `Target_DB.payments` p
INNER JOIN `Target_DB.orders` o
  ON o.order_id = p.order_id

-- ignore cancelled and processing orders
WHERE o.order_status = 'delivered'
GROUP BY 1
ORDER BY 1 ;
```

Row	ym	UPI	Credit_card	Debit_card	Voucher
1	2016-10-01	51	209	2	20
2	2016-12-01	0	1	0	0
3	2017-01-01	188	542	9	60
4	2017-02-01	371	1257	13	108
5	2017-03-01	565	1908	30	197
6	2017-04-01	474	1772	25	165
7	2017-05-01	740	2733	29	285
8	2017-06-01	689	2373	26	235
9	2017-07-01	811	2974	20	342
10	2017-08-01	902	3186	33	272
11	2017-09-01	868	3183	43	277
12	2017-10-01	955	3416	51	276

Insight Query :

```
SELECT
  ROUND(SUM(UPI) / SUM(total_order) * 100, 2) AS perc_UPI,
  ROUND(SUM(Credit_card) / SUM(total_order) * 100, 2) AS perc_credit_card,
  ROUND(SUM(Debit_card) / SUM(total_order) * 100, 2) AS perc_debit_card,
  ROUND(SUM(Voucher) / SUM(total_order) * 100, 2) AS perc_voucher
FROM final_table;
```

Row	perc_UPI	perc_credit_card	perc_debit_card	perc_voucher
1	19.05	74.03	1.47	5.45

2. Find the no. of orders placed on the basis of the payment installments that have been paid.

```
SELECT COUNT(*) AS total_orders_paid_in_installments
FROM
  (SELECT order_id
   FROM `Target_DB.payments`
   GROUP BY order_id
   HAVING MAX(payment_installments) > 1) t;
```

Row	total_orders_paid_in_installments
1	51170

Key Insights :

On MoM number of orders placed using different payment types and orders placed on the basis of the payment installments

- ~51% of orders are paid in installments, showing that customers prefer EMI payments rather than paying upfront
- Delivered orders payment type demographics
 - 74% - Credit Card
 - 19% - UPI
 - 1.5% - Debit card
 - 5.5% Voucher

Most of the orders are placed using Credit card , UPI and Vouchers which all **comes under payment methods that include some kind of cash back or offers or coupon code.**

In other words,

- More than 98% of orders come via payment modes linked to cash back, offers, or coupons.
- 51% of the orders are paid in EMI which is done through Credit cards indicating strong reliance on bank-driven EMI programs.

Action Recommendations :

- Marketing team should focus on campaigns that offer such cash back, EMI option and discount offers.
 - Marketing and Finance teams can consider collaborations with banks, UPI apps, and voucher providers to create exclusive deals.
 - Reconsider debit card strategy : With such low adoption ~1%, investing heavily in debit card offers may not be ROI-positive.
-

Section 7 :

Insights at a Glance :

YoY Analysis :

- From 2017-2018 there is a ~19% increase in orders and 2018 only has 10 active months with last month being 2018-10-17

MoM analysis :

- September and December shows significant drop in sales across both MoM growth and average per month.
- March, May, July, November Shows atleast 10% increase in orders across both MoM growth and average per month.
- November has the highest sales increase with 350% MoM growth and 74% higher than average.

MoM Order per State Insights :

- There is a total of 565 states with monthly orders from 2016-2018
 - Bottom 250+ of them have order count less than 50
Which means in that state customers placed less than 50 orders in a month.
 - Top 45 have order count more than 500

There is a big deviation between the top and the bottom state orders per month. So, I have categorised them into two groups.

- "Whales" the states-month with orders **more than 500 per month** :

- They are top 7% of the total state-month population.
- They contribute to 60% of the total orders
- "small fish" the states-month with orders **less than 50 per month** :
 - They are bottom 56% of the total state-month population.
 - They contribute to 6% of the total orders

State wise customers Insights :

- State **SP** has the highest customer base of ~37%
- Top 10 states out of 27 contribute to ~85% of the customers.
- The bottom 10 states contribute to <5% of the customers.
- Classic 80/20 distribution. Top 6 states contribute to ~80% of customers.
- Rank 11-17 contribute to ~9% and can be potential opportunity zones.
- State **SP** has the highest customer base of ~37% and the lowest average freight value 17.37 per order.

Delivery time Insights :

- Average delivery time of an order is 12 days.
- Average estimated delivery time of an order is 23 days
- 90% of orders are delivered early and they are early by ~11 days.
- 3% of orders are delivered on the estimated date.
- 7% orders are delivered late by ~13 days from the estimated date.

Delivery speed, Freight Insights :

- By Comparing State's delivery time, freight cost and Order data. We find out inconsistencies in logistics performance among high volume states.
Eg : State **RS** contributes ~5.5% of the total orders in Brazil but does not appear in top 5 fastest delivering states or lowest freight states.
- High Order Volume Vs Delivery speed misalignment
Not one of the top 10 high-volume order contributors are in the top 5 fastest delivering states.

Payments Insights :

- Half of all orders rely on EMI — incentives payment options which is central to customer buying decisions.